

JavaScript

Damian Nicolalde-R.
danicolalde@puce.edu.ec

Temas:

- **Introducción a JavaScript**
- **Funciones**
- Objetos
- Objetos estándar
- Eventos
- Depuración
- API

Introducción a JavaScript

- ***HTML y CSS incluyen instrucciones para indicar al navegador cómo debe organizar y visualizar un documento y su contenido.*** Podemos crear un formulario con campos de entrada, controles y botones, pero HTML solo provee la funcionalidad necesaria para enviar la información introducida por el usuario al servidor o para limpiar el formulario. Algo similar pasa con CSS; podemos construir instrucciones (reglas) con seudoclases como :hover para aplicar un grupo diferente de propiedades cuando el usuario mueve el ratón sobre un elemento, pero si queremos realizar tareas personalizadas, como modificar los estilos de varios elementos al mismo tiempo, debemos cargar una nueva hoja de estilo que ya presente estos cambios. **Con el propósito de alterar elementos de forma dinámica, realizar operaciones personalizadas, o responder al usuario y a cambios que ocurren en el documento, los navegadores incluyen un tercer lenguaje llamado JavaScript.**
- ***JavaScript es un lenguaje de programación que se usa para procesar información y manipular documentos.*** Al igual que cualquier otro lenguaje de programación, JavaScript provee instrucciones que se ejecutan de forma secuencial para indicarle al sistema lo que queremos que haga (realizar una operación aritmética, asignar un nuevo valor a un elemento, etc.). Cuando el navegador encuentra este tipo de código en nuestro documento, ejecuta las instrucciones al momento y cualquier cambio realizado en el documento se muestra en pantalla.

Implementando JavaScript

- Siguiendo el mismo enfoque que CSS, el código JavaScript se puede incorporar al documento mediante tres técnicas diferentes: el código se puede insertar en un elemento por medio de atributos (En línea), incorporar al documento como contenido del elemento `<script>` o cargar desde un archivo externo. La técnica En línea aprovecha atributos especiales que describen un evento, como un clic del ratón. Para lograr que un elemento responda a un evento usando esta técnica, todo lo que tenemos que hacer es agregar el atributo correspondiente con el código que queremos que se ejecute.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4  |   <meta charset="utf-8">
5  |   <title>JavaScript</title>
6  </head>
7  <body>
8  |   <section>
9  | |   <p onclick="alert('Hizo clic!')">Clic aquí</p>
10 | |   <p>No puede hacer clic aquí</p>
11 |   </section>
12 </body>
13 </html>
```

El atributo onclick agregado al elemento <p> dice algo similar a «cuando alguien hace clic en este elemento, ejecutar este código», y el código es (en este caso) la instrucción alert(). Esta es una instrucción predefinida en JavaScript llamada *función*. Lo que esta función hace es mostrar una ventana emergente con el valor provisto entre paréntesis. Cuando el usuario hace clic en el área ocupada por el elemento <p>, el navegador ejecuta la función alert() y muestra una ventana emergente en la pantalla con el mensaje «Hizo clic!».

Implementando JavaScript

El atributo onclick es parte de una serie de atributos provistos por HTML para responder a eventos, los siguientes son los atributos más usados asociados con el ratón.

- **onclick**—Este atributo responde al evento click. El evento se ejecuta cuando el usuario hace clic con el botón izquierdo del ratón. HTML ofrece otros dos atributos similares llamados ondblclick (el usuario hace doble clic con el botón izquierdo del ratón) y oncontextmenu (el usuario hace clic con el botón derecho del ratón).
- **onmousedown**—Este atributo responde al evento mousedown. Este evento se desencadena cuando el usuario pulsa el botón izquierdo o el botón derecho del ratón.
- **onmouseup**—Este atributo responde al evento mouseup. El evento se desencadena cuando el usuario libera el botón izquierdo del ratón.
- **onmouseenter**—Este atributo responde al evento mouseenter. Este evento se desencadena cuando el ratón se introduce en el área ocupada por el elemento.

Implementando JavaScript

- **onmouseleave**—Este atributo responde al evento mouseleave. Este evento se desencadena cuando el ratón abandona el área ocupada por el elemento.
- **onmouseover**—Este atributo responde al evento mouseover. Este evento se desencadena cuando el ratón se mueve sobre el elemento o cualquiera de sus elementos hijos.
- **onmouseout**—Este atributo responde al evento mouseout. El evento se desencadena cuando el ratón abandona el área ocupada por el elemento o cualquiera de sus elementos hijos.
- **onmousemove**—Este atributo responde al evento mousemove. Este evento se desencadena cada vez que el ratón se encuentra sobre el elemento y se mueve.
- **onwheel**—Este atributo responde al evento wheel. Este evento se desencadena cada vez que se hace girar la rueda del ratón.

Implementando JavaScript

Los siguientes son los atributos disponibles para responder a eventos generados por el teclado. Estos tipos de atributos se aplican a elementos que aceptan una entrada del usuario, como los elementos `<input>` y `<textarea>`.

- **onkeypress**—Este atributo responde al evento `keypress`. Este evento se desencadena cuando se activa el elemento y se pulsa una tecla.
- **onkeydown**—Este atributo responde al evento `keydown`. Este evento se desencadena cuando se activa el elemento y se pulsa una tecla.
- **onkeyup**—Este atributo responde al evento `keyup`. Este evento se desencadena cuando se activa el elemento y se libera una tecla.

También contamos con otros dos atributos importantes asociados al documento:

- **onload**—Este atributo responde al evento `load`. El evento se desencadena cuando un recurso termina de cargarse.
- **onunload**—Este atributo responde al evento `unload`. Este evento se desencadena cuando un recurso termina de cargarse.

Implementando JavaScript

- Los atributos de evento se incluyen en un elemento dependiendo de cuándo queremos que se ejecute el código. Si queremos responder al clic del ratón, tenemos que incluir el atributo onclick, como hemos hecho en el ejemplo anterior, pero si queremos iniciar un proceso cuando el puntero del ratón pasa sobre un elemento, tenemos que incluir los atributos onmouseover u onmousemove.
- Debido a que en un elemento pueden ocurrir varios eventos en algunos casos al mismo tiempo, podemos declarar más de un atributo por cada elemento. Por ejemplo, el siguiente documento incluye un elemento <p> con dos atributos, onclick y onmouseout, que incluyen sus propios códigos JavaScript. Si el usuario hace clic en el elemento, se muestra una ventana emergente con el mensaje «Hizo clic!», pero si el usuario mueve el ratón fuera del área ocupada por el elemento, se muestra una ventana emergente diferente con el mensaje «No me abandone!».

```
1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>JavaScript</title>
6          <style>
7              p{
8                  background-color:  aqua;
9              }
10         </style>
11     </head>
12     <body>
13         <section>
14             <p onclick="alert('Hizo clic!')" onmouseout="alert('No me abandone!')">Clic aquí</p>
15         </section>
16     </body>
17 </html>
```

Implementando JavaScript

- Los eventos no solo los produce el usuario, sino también el navegador. Un evento útil desencadenado por el navegador es load. Este evento se desencadena cuando se ha terminado de cargar un recurso y, por lo tanto, se utiliza frecuentemente para ejecutar código JavaScript después de que el navegador ha cargado el documento y su contenido.

```
1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>JavaScript</title>
6      </head>
7      <body onload="alert('Bienvenido!')">
8          <section>
9              <h1>Mi Sitio Web</h1>
10             <p>Bienvenido a mi sitio web</p>
11          </section>
12      </body>
13 </html>
```

Muestra una ventana emergente para dar la bienvenida al usuario después de que se ha cargado completamente. El navegador primero carga el contenido del documento y cuando termina, llama a la función `alert()` y muestra el mensaje en la pantalla.

Implementando JavaScript

- Los atributos de evento son útiles cuando queremos probar código o implementar una función de inmediato, pero no son apropiados para aplicaciones importantes. Para trabajar con códigos extensos y personalizar las funciones, tenemos que agrupar el código con el elemento `<script>`. El elemento `<script>` actúa igual que el elemento `<style>` para CSS, organizando el código en un solo lugar y afectando al resto de los elementos en el documento usando referencias.

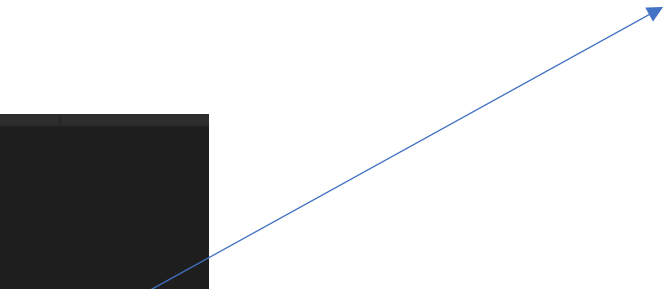
```
1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>JavaScript</title>
6          <script>
7              alert('Bienvenido!');
8          </script>
9      </head>
10     <body>
11         <section>
12             <p>Hola</p>
13         </section>
14     </body>
15 </html>
```

- El elemento `<script>` y su contenido se pueden ubicar en cualquier parte del documento, pero normalmente se introducen dentro de la cabecera, como hemos hecho en este ejemplo.
- De esta manera, cuando el navegador carga el archivo, lee el contenido del elemento `<script>`, ejecuta el código al instante, y luego continúa procesando el resto del documento.

Implementando JavaScript

- Introducir JavaScript en el documento con el elemento `<script>` puede resultar práctico cuando tenemos un grupo pequeño de instrucciones, pero el código JavaScript crece con rapidez en aplicaciones profesionales. Si usamos el mismo código en más de un documento, tendremos que mantener diferentes versiones del mismo programa y los navegadores tendrán que descargar el mismo código una y otra vez con cada documento solicitado por el usuario. Una alternativa es introducir el código JavaScript en un archivo externo y luego cargarlo desde los documentos que lo requieren. De esta manera, solo los documentos que necesitan ese grupo de instrucciones deberán incluir el archivo, y el navegador tendrá que descargar el archivo una sola vez (los navegadores mantienen los archivos en un caché en el ordenador del usuario en caso de que sean requeridos más adelante por otros documentos del mismo sitio web). Para este propósito, el elemento `<script>` incluye el atributo `src`. Con este atributo, podemos declarar la ruta al archivo JavaScript y escribir todo nuestro código dentro de este archivo.

```
1 <!DOCTYPE html>
2 <html lang="es">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript</title>
6     <script src="micodigo1.js"></script>
7   </head>
8   <body>
9     <section>
10      <p>Hola</p>
11    </section>
12  </body>
13 </html>
```



```
1 alert("Bienvenido!");
```

Al igual que los archivos HTML y CSS, los archivos JavaScript son simplemente archivos de texto que podemos crear con cualquier editor de texto o los editores profesionales. A estos tipos de archivos se les puede asignar cualquier nombre, pero por convención tienen que tener la extensión .js. El archivo debe contener el código JavaScript exactamente según se declara entre las etiquetas <script>. Por ejemplo, el código que tenemos que introducir en el archivo micodigo1.js para reproducir el ejemplo anterior.

Implementando JavaScript

Lo básico: en JavaScript se recomienda finalizar cada instrucción con un punto y coma para asegurarnos de que el navegador no tenga ninguna dificultad al identificar el final de cada instrucción. El punto y coma se puede ignorar, pero nos puede ayudar a evitar errores cuando el código está compuesto por múltiples instrucciones, como ocurre frecuentemente.

IMPORTANTE: además del atributo `src`, el elemento `<script>` también puede incluir los atributos `async` y `defer`. Estos son atributos booleanos que indican cómo y cuándo se debe ejecutar el código. Si el atributo `async` se declara, el código se ejecuta de forma asíncrona (mientras se procesa el resto del documento). Si el atributo `defer` se declara en su lugar, el código se ejecuta después de que el documento completo se haya procesado.

Variables

- Por supuesto, JavaScript es algo más que ventanas emergentes mostrando mensajes para alertar al usuario. El lenguaje puede realizar numerosas tareas, desde calcular algoritmos complejos hasta procesar el contenido de un documento. Cada una de estas tareas involucra la manipulación de valores, y esta es la razón por la que la característica más importante de JavaScript, al igual que cualquier otro lenguaje de programación, es la capacidad de almacenar datos en memoria.
- La memoria de un ordenador o dispositivo móvil es como un panal de abejas con millones y millones de celdas consecutivas en las que se almacena información. Estas celdas tienen un espacio limitado y, por lo tanto, es necesaria la combinación de múltiples celdas para almacenar grandes cantidades de datos. Debido a la complejidad de esta estructura, los lenguajes de programación incorporan el concepto de *variables* para facilitar la identificación de cada valor almacenado en memoria. Las variables son simplemente nombres asignados a una celda o un grupo de celdas donde se van a almacenar los datos. Por ejemplo, si almacenamos el valor 5, tenemos que saber en qué parte de la memoria se encuentra para poder leerlo más adelante. La creación de una variable nos permite identificar ese espacio de memoria con un nombre y usar ese nombre más adelante para leer el valor o reemplazarlo por otro.
- Las variables en JavaScript se declaran con la palabra clave `var` seguida del nombre que queremos asignarle. Si queremos almacenar un valor en el espacio de memoria asignado por el sistema a la variable, tenemos que incluir el carácter `=` (igual) seguido del valor, como en el siguiente ejemplo.

```
var minumero = 2;
```

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo2.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```

```
1  var numero=2;
2  alert("el numero almacenado en memoria es:" + numero);
```

Como ya hemos mencionado, las instrucciones en un programa JavaScript las ejecuta el navegador una por una en secuencia. Por lo tanto, cuando el navegador lee el código del ejemplo, ejecuta las instrucciones de arriba abajo

Variables

- Las variables se denominan así porque sus valores no son constantes. Podemos cambiar sus valores cada vez que lo necesitemos, y esa es, de hecho, su característica más importante.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo3.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```

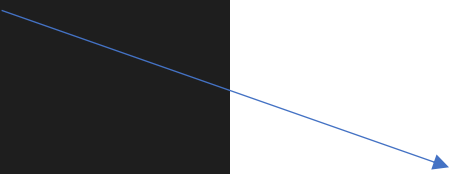
```
1  var numero=2;
2  alert("el numero almacenado en memoria es:" + numero);
3  numero=2000;
4  alert("el numero almacenado en memoria modificado es:" + numero);
```



Variables

- El valor almacenado en una variable se puede asignar a otra. Por ejemplo, el siguiente código crea dos variables llamadas `numero` y `tunumero`, y asigna el valor almacenado en la variable `numero` a la variable `tunumero`. El valor mostrado en la pantalla es el número 2.

```
1 <!DOCTYPE html>
2 <html lang="es">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript</title>
6     <script src="micodigo4.js"></script>
7   </head>
8   <body>
9     <section>
10      <p>Hola</p>
11    </section>
12  </body>
13 </html>
```



```
1 var numero=2;
2 var tunumero=numero;
3 alert("el numero almacenado en memoria es:" + tunumero);
```

Variables

- En una situación más práctica, probablemente usaríamos el valor de la variable para ejecutar una operación y asignar el resultado de vuelta a la misma variable.

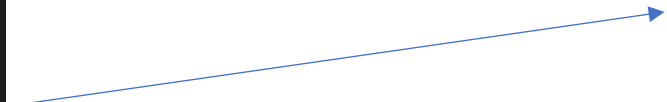
```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4    <meta charset="utf-8">
5    <title>JavaScript</title>
6    <script src="micodigo5.js"></script>
7  </head>
8  <body>
9    <section>
10     <p>Hola</p>
11   </section>
12 </body>
13 </html>
```

```
1  var numero=2;
2  numero=numero+3;
3  alert("el numero almacenado en memoria y sumado 3 es:" + numero);
4  //comentario una linea
5
6  /*
7   comentarios varias lineas
8  */
```

Variables

- Además del operador +, JavaScript también incluye los operadores - (resta), * (multiplicación), / (división), y % (módulo). Estos operadores se pueden usar en una operación sencilla entre dos valores o combinados con múltiples valores para realizar operaciones aritméticas más complejas.

```
1  <!DOCTYPE html>
2  <html lang="es">
3    <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo6.js"></script>
7    </head>
8    <body>
9      <section>
10       <p>Hola</p>
11     </section>
12   </body>
13 </html>
```

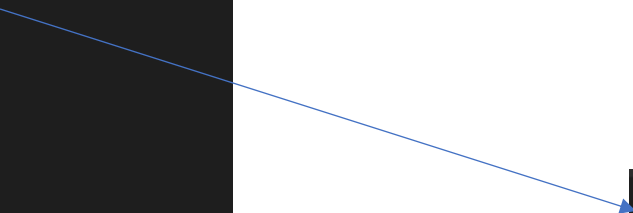


```
1  var minumero = 2;
2  minumero = minumero * 25 + 3; // 53
3  alert(minumero);
```

Variables

- Las operaciones aritméticas se ejecutan siguiendo un orden de prioridad determinado por los operadores. La multiplicación y la división tienen prioridad sobre la adición y la sustracción. Esto significa que las multiplicaciones y divisiones se calcularán antes que las sumas y restas. En el ejemplo anterior, el valor actual de la variable `minumero` se multiplica por 25, y luego el valor 3 se suma al resultado. Si queremos controlar la precedencia, podemos aislar las operaciones con paréntesis. Por ejemplo, el siguiente código realiza la adición primero y luego la multiplicación, lo que genera un resultado diferente.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  |   <head>
4  |   |   <meta charset="utf-8">
5  |   |   <title>JavaScript</title>
6  |   |   <script src="micodigo7.js"></script>
7  |   </head>
8  |   <body>
9  |   |   <section>
10 |   |   |   <p>Hola</p>
11 |   |   </section>
12 |   </body>
13 </html>
```



```
1  var minumero = 2;
2  minumero = minumero * (25 + 3); // 56
3  alert(minumero);
```

Variables

Realizar una operación en el valor actual de una variable y asignar el resultado de vuelta a la misma variable es muy común en programación. JavaScript ofrece los siguientes operadores para simplificar esta tarea.

- `++` es una abreviatura de la operación `variable = variable + 1`.
- `--` es una abreviatura de la operación `variable = variable - 1`.
- `+=` es una abreviatura de la operación `variable = variable + number`.
- `-=` es una abreviatura de la operación `variable = variable - number`.
- `*=` es una abreviatura de la operación `variable = variable * number`.
- `/=` es una abreviatura de la operación `variable = variable / number`.

Con estos operadores, podemos realizar operaciones en los valores de una variable y asignar el resultado de vuelta a la misma variable. Un uso común de estos operadores es el de crear contadores que incrementan o disminuyen el valor de una variable en una o más unidades. Por ejemplo, el operador `++` suma el valor 1 al valor actual de la variable cada vez que se ejecuta la instrucción

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo8.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```



```
1  var minumero = 0;
2  minumero++;
3  alert(minumero); // 1
```

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo9.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```

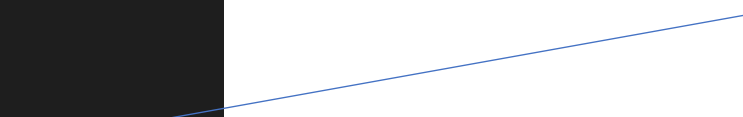


```
1  var minumero = 0;
2  minumero += 5;
3  alert(minumero); // 5
```

Cadenas de texto

- En los anteriores ejemplos hemos almacenado números, pero las variables también se pueden usar para almacenar otros tipos de valores, incluido texto. Para asignar texto a una variable, tenemos que declararlo entre comillas simples o dobles.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo10.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```

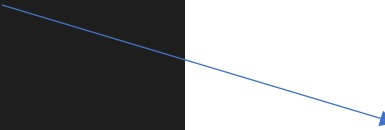


```
1  var mitexto = "Hola Mundo!";
2  alert(mitexto);
```

Cadenas de texto

- El espacio reservado en memoria por el sistema para almacenar la cadena de caracteres depende del tamaño del texto (cuántos caracteres contiene), pero el sistema está preparado para ampliar este espacio si luego se asignan valores más extensos a la variable. Una situación común en la cual se asignan textos más extensos a la misma variable es cuando agregamos más caracteres al comienzo o al final del valor actual de la variable. El texto se puede concatenar con el operador +.

```
1  <!DOCTYPE html>
2  <html lang="es">
3    <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo11.js"></script>
7    </head>
8    <body>
9      <section>
10       <p>Hola</p>
11     </section>
12   </body>
13 </html>
```



```
1  var minombre = "Mi nombre es ";
2  var miapellido=" Mi apellido es "
3  minombre = minombre + "Damian" + miapellido + "Nicolalde";
4  alert(minombre);
```

Cadenas de texto

- Si en lugar de texto intentamos concatenar una cadena de caracteres con un número, el número se convierte en una cadena de caracteres y se agrega al valor actual. El siguiente código produce la cadena de caracteres "El número es 3".

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo12.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```



```
1  var mitexto = "El número es " + 3;
2  alert(mitexto);
```

Cadenas de texto

- Este procedimiento es importante cuando tenemos una cadena de caracteres que contiene un número y queremos agregarle otro número. Debido a que JavaScript considera el valor actual como una cadena de caracteres, el número también se convierte en una cadena de caracteres y los valores no se suman.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo13.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```

```
1  var mitexto = "20" + 3;
2  alert(mitexto); // "203"
```

el resultado del código no es 23, sino la cadena de caracteres "203". El sistema convierte el número 3 en una cadena de caracteres y concatena las cadenas en lugar de sumar los números.

Cadenas de texto

- Las cadenas de caracteres pueden contener cualquier carácter que queramos, y esto incluye comillas simples o dobles. Si las comillas en el texto son diferentes a las comillas usadas para definir la cadena de caracteres, estas se tratan como cualquier otro carácter, pero si las comillas son las mismas, el sistema no sabe dónde termina el texto. Para resolver este problema, JavaScript ofrece el carácter de escape \. Por ejemplo, si la cadena de caracteres se ha declarado con comillas simples, tenemos que escapar las comillas simples dentro del texto.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo14.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```



```
1  var mitexto = 'El \'libro\' es interesante';
2  alert(mitexto); // "El 'libro' es interesante"
```

Cadenas de texto

- JavaScript ofrece varios caracteres de escape con diferentes propósitos. Los que se utilizan con más frecuencia son `\n` para generar una nueva línea y `\r` para devolver el cursor al comienzo de la línea. Generalmente, estos dos caracteres se implementan en conjunto para dividir el texto en múltiples líneas, tal como muestra el siguiente ejemplo.

```
1 <!DOCTYPE html>
2 <html lang="es">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript</title>
6     <script src="micodigo15.js"></script>
7   </head>
8   <body>
9     <section>
10      <p>Hola</p>
11    </section>
12  </body>
13 </html>
```



```
1 var mitexto = "Felicidad no es hacer lo que uno quiere\r\n";
2 mitexto = mitexto + "sino querer lo que uno hace."
3 alert(mitexto);
```

Booleanos

- Otro tipo de valores que podemos almacenar en variables son los booleanos. Las variables booleanas pueden contener solo dos valores: true (verdadero) o false (falso). Estas variables son particularmente útiles cuando solo necesitamos determinar el estado actual de una condición.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo16.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```

```
1  var valido = true;
2  alert(valido);
```

El propósito de estas variables es el de simplificar el proceso de identificación del estado de una condición. Si usamos un número entero para indicar un estado, deberemos recordar qué números decidimos usar para representar los estados válido y no válido. Usando valores booleanos en su lugar, solo tenemos que comprobar si el valor es igual a true o false.

Arrays

- Las variables también pueden almacenar varios valores al mismo tiempo en una estructura llamada *array*. Los arrays se pueden crear usando una sintaxis simple que incluye los valores separados por comas dentro de corchetes. Los valores se identifican luego mediante un índice, comenzando desde 0 (cero).

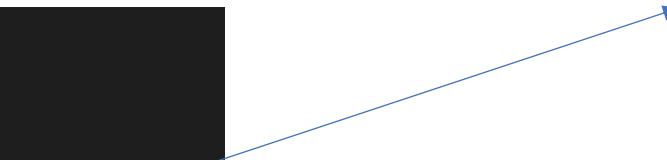
```
1  <!DOCTYPE html>
2  <html lang="es">
3    <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo17.js"></script>
7    </head>
8    <body>
9      <section>
10       <p>Hola</p>
11     </section>
12   </body>
13 </html>
```

```
1  var miarray = ["rojo", "verde", "azul"];
2  alert(miarray[0]); // "rojo"
```

Arrays

- La función `alert()` puede mostrar no solo valores independientes, sino arrays completos. Si queremos ver todos los valores incluidos en el array, solo tenemos que especificar el nombre del array.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo18.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```



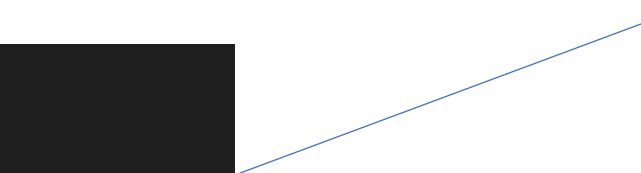
```
1  var miarray = ["rojo", "verde", "azul"];
2  alert(miarray); // "rojo,verde,azul"
```

Arrays

- Los arrays, al igual que cualquier otra variable, pueden contener cualquier tipo de valor que deseemos. Por ejemplo, podemos crear un array combinando números y cadenas de caracteres.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo19.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```

```
1  var miarray = ["rojo", 32, "HTML5 es genial!"];
2  alert(miarray);
```



Arrays

Si intentamos leer un valor en un índice que aún no se ha definido, JavaScript devuelve el valor undefined (indefinido) Este valor lo usa el sistema para informar de que el valor que estamos intentando acceder no existe, pero también podemos asignarlo a un array cuando aún no contamos con el valor para esa posición.

```
var miarray = ["rojo", undefined, 32];  
alert(miarray[1]);
```

Otra manera mejor de indicarle al sistema que no existe un valor disponible en un momento para un índice del array es usando otro valor especial llamado null (nulo). La diferencia entre los valores undefined y null es que undefined indica que la variable fue declarada pero ningún valor le fue asignado, mientras que null indica que existe un valor, pero es nulo.

```
var miarray = ["rojo", 32, null];  
alert(miarray[2]);
```

Arrays

- Por supuesto, también podemos realizar operaciones en los valores de un array y almacenar los resultados, como hemos hecho antes con variables sencillas.

```
1  var miarray = [64, 32];  
2  miarray[1] = miarray[1] + 10;  
3  alert("El valor actual es " + miarray[1]); // "El valor actual es 42"
```

```
1  <!DOCTYPE html>  
2  <html lang="es">  
3      <head>  
4          <meta charset="utf-8">  
5          <title>JavaScript</title>  
6          <script src="micodigo20.js"></script>  
7      </head>  
8      <body>  
9          <section>  
10             <p>Hola</p>  
11          </section>  
12      </body>  
13 </html>
```

Los arrays trabajan exactamente igual que otras variables, con la excepción de que tenemos que mencionar el índice cada vez que queremos usarlos.

Arrays

- Los arrays pueden incluir cualquier tipo de valores, por lo que es posible declarar arrays de arrays. Estos tipos de arrays se denominan *arrays multidimensionales*.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo21.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```

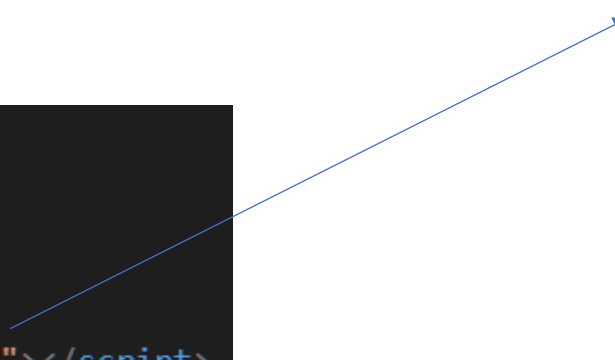
```
1  var miarray = [[2, 45, 31], [5, 10], [81, 12]];
2  alert(miarray[1][0]); // 5
```

El ejemplo se crea un array de arrays de números enteros. Para acceder a estos valores, tenemos que declarar los índices de cada nivel entre corchetes, uno después del otro. El ejemplo devuelve el primer valor (índice 0) del segundo array (índice 1). La instrucción busca el array en el índice 1 y luego busca por el número en el índice 0.

Arrays

- Si queremos eliminar uno de los valores, podemos declararlo como undefined o null, como hemos hecho anteriormente, o declararlo como un array vacío asignando corchetes sin valores en su interior.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo22.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```

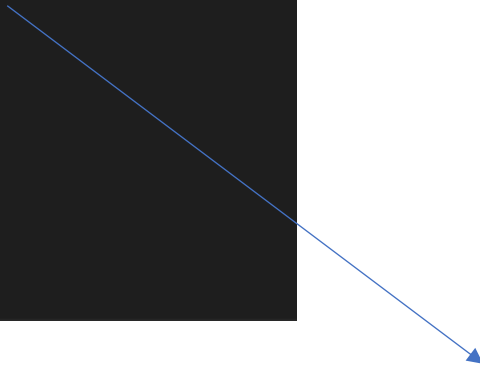


```
1  var miarray = [[2, 45, 31], [5, 10], [81, 12]];
2  miarray[1] = []
3  alert(miarray[1][0]); // undefined
```

Condicionales y bucles

- Hasta este punto hemos escrito instrucciones en secuencia, una debajo de la otra. En este tipo de programas, el sistema ejecuta cada instrucción una sola vez y en el orden en el que se presentan. Comienza con la primera y sigue hasta llegar al final de la lista. El propósito de condicionales y bucles es el de romper esta secuencia. Los condicionales nos permiten ejecutar una o más instrucciones solo cuando se cumple una determinada condición, y los bucles nos permiten ejecutar un bloque de código (un grupo de instrucciones) una y otra vez hasta que se satisface una condición. JavaScript ofrece un total de cuatro instrucciones para procesar código de acuerdo a condiciones determinadas por el programador: **if**, **switch**, **for** y **while**.
- La manera más simple de comprobar una condición es con la instrucción **if**. Esta instrucción analiza una expresión y procesa un grupo de instrucciones si la condición establecida por esa expresión es verdadera. La instrucción requiere la palabra clave **if** seguida de la condición entre paréntesis y las instrucciones que queremos ejecutar si la condición es verdadera entre llaves.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo23.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```



```
1  var mivariable = 9;
2  if (mivariable < 10) {
3      alert("El numero "+ mivariable +" es menor que 10");
4  }
```

Condicionales y bucles

Los siguientes son los operadores de comparación disponibles en JavaScript.

- `==` comprueba si el valor de la izquierda es igual al de la derecha.
- `!=` comprueba si el valor de la izquierda es diferente al de la derecha.
- `>` comprueba si el valor de la izquierda es mayor que el de la derecha.
- `<` comprueba si el valor de la izquierda es menor que el de la derecha.
- `>=` comprueba si el valor de la izquierda es mayor o igual que el de la derecha.
- `<=` comprueba si el valor de la izquierda es menor o igual que el de la derecha.

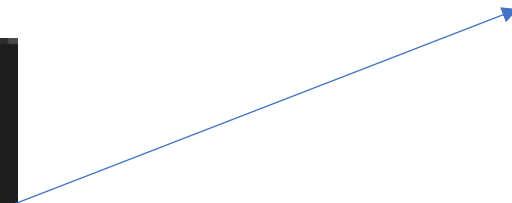
Después de evaluar una condición, esta devuelve un valor lógico verdadero o falso. Esto nos permite trabajar con condiciones como si fueran valores y combinarlas para crear condiciones más complejas. JavaScript ofrece los siguientes operadores lógicos con este propósito.

- `!` (negación) permuta el estado de la condición. Si la condición es verdadera, devuelve falso, y viceversa.
- `&&` (y) comprueba dos condiciones y devuelve verdadero si ambas son verdaderas.
- `||` (o) comprueba dos condiciones y devuelve verdadero si una o ambas son verdaderas.

Condicionales y bucles

- El operador lógico ! invierte el estado de la condición. Si la condición se evalúa como verdadera, el estado final será falso, y las instrucciones entre llaves no se ejecutarán.

```
1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>JavaScript</title>
6          <script src="micodigo24.js"></script>
7      </head>
8      <body>
9          <section>
10             <p>Hola</p>
11          </section>
12      </body>
13 </html>
```

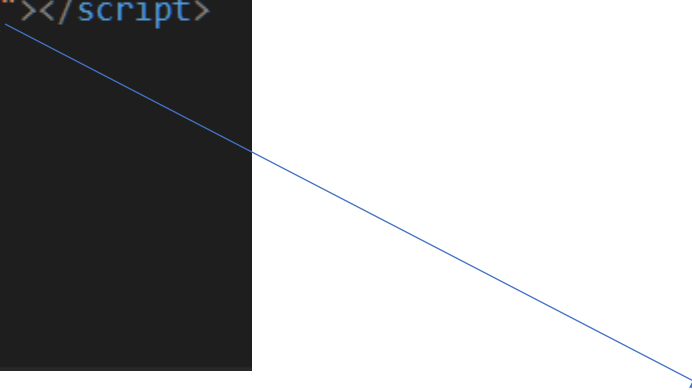


```
1  var mivariable = 9;
2  if (!(mivariable < 10)) {
3      alert("El número es menor que 10");
4  }
```

Condicionales y bucles

- El código anterior no muestra ningún mensaje en la pantalla. El valor 9 es aún menor que 10, pero debido a que alteramos la condición con el operador `!`, el resultado final es falso, y la función `alert()` no se ejecuta.
- Para que el operador trabaje sobre el estado de la condición y no sobre los valores que estamos comparando, debemos encerrar la condición entre paréntesis. Debido a los paréntesis, la condición se evalúa en primer lugar y luego el estado que devuelve se invierte con el operador `!`.
- Los operadores `&&` (y) y `||` (o) trabajan de un modo diferente. Estos operadores calculan el resultado final basándose en los resultados de las condiciones involucradas. El operador `&&` (y) devuelve verdadero solo si las condiciones a ambos lados devuelven verdadero, y el operador `||` (o) devuelve verdadero si una o ambas condiciones devuelven verdadero. Por ejemplo, el siguiente código ejecuta la función `alert()` solo cuando la edad es menor de 21 y el valor de la variable inteligente es igual a "SI" (debido a que usamos el operador `&&`, ambas condiciones tienen que ser verdaderas para que la condición general sea verdadera).

```
1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>JavaScript</title>
6          <script src="micodigo25.js"></script>
7      </head>
8      <body>
9          <section>
10             <p>Hola</p>
11          </section>
12      </body>
13 </html>
```

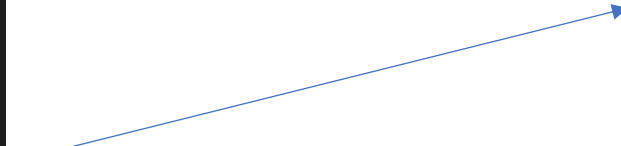


```
1  var inteligente = "SI";
2  var edad = 19;
3  if (edad < 21 && inteligente == "SI") {
4      alert("Juan está autorizado");
5  }
```

Condicionales y bucles

- Si asumimos que nuestro ejemplo solo considera dos valores para la variable inteligente, "SI" y "NO", podemos convertirla en una variable booleana. Debido a que los valores booleanos son valores lógicos, no necesitamos compararlos con nada. El siguiente código simplifica el ejemplo anterior usando una variable booleana.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo26.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```



```
1  var inteligente = true;
2  var edad = 19;
3  if (edad < 21 && inteligente) {
4      alert("Juan está autorizado");
5  }
```

Condicionales y bucles

- JavaScript es bastante flexible en cuanto a los valores que podemos usar para establecer condiciones. El lenguaje es capaz de determinar una condición basándose en los valores de cualquier variable. Por ejemplo, una variable con un número entero devolverá falso si el valor es 0 o verdadero si el valor es diferente de 0.

```
var edad = 0;  
if (edad) {  
    alert("Juan está autorizado");  
}
```

El código de la instrucción if del ejemplo no se ejecuta porque el valor de la variable edad es 0 y, por lo tanto, el estado de la condición se considera falso. Si almacenamos un valor diferente dentro de esta variable, la condición será verdadera y el mensaje se mostrará en la pantalla.

Condicionales y bucles

- Las variables con cadenas de caracteres vacías también devuelven falso. El siguiente ejemplo comprueba si se ha asignado una cadena de caracteres a una variable y muestra su valor solo si la cadena no está vacía.

```
var nombre = "Juan";  
if (nombre) {  
    alert(nombre + " está autorizado");  
}
```

Condicionales y bucles

- A veces debemos ejecutar instrucciones para cada estado de la condición (verdadero o falso). JavaScript incluye la instrucción `if else` para ayudarnos en estas situaciones. Las instrucciones se presentan en dos bloques de código delimitados por llaves. El bloque precedido por `if` se ejecuta cuando la condición es verdadera y el bloque precedido por `else` se ejecuta en caso contrario.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo27.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```



```
1  var mivariable = 2;
2  if (mivariable < 10) {
3      alert("El número es menor que 10");
4  } else {
5      alert("El numero es igual o mayor que 10");
6  }
```

Condicionales y bucles

- Si lo que necesitamos es comprobar múltiples condiciones, en lugar de las instrucciones if else podemos usar la instrucción switch. Esta instrucción evalúa una expresión (generalmente una variable), compara el resultado con múltiples valores y ejecuta las instrucciones correspondientes al valor que coincide con la expresión. La sintaxis incluye la palabra clave switch seguida de la expresión entre paréntesis. Los posibles valores se listan usando la palabra clave case, tal como muestra el siguiente ejemplo.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo28.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```

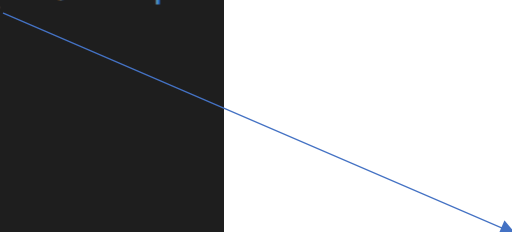


```
1  var mivariable = 8;
2  switch(mivariable) {
3      case 5:
4          alert("El número es cinco");
5          break;
6      case 8:
7          alert("El número es ocho");
8          break;
9      case 10:
10         alert("El número es diez");
11         break;
12     default:
13         alert("El número es " + mivariable);
14 }
```

Condicionales y bucles

- En JavaScript, una vez que se encuentra una coincidencia, las instrucciones en ese caso se ejecutan junto con las instrucciones de los casos siguientes. Este es el comportamiento por defecto, pero normalmente no lo que nuestro código necesita. Por esta razón, JavaScript incluye la instrucción `break`. Para evitar que el sistema ejecute las instrucciones de cada caso después de que se encuentra una coincidencia, tenemos que incluir la instrucción `break` al final de cada caso.
- A veces tenemos que ejecutar las instrucciones varias veces para la misma condición o evaluar la condición nuevamente cada vez que se termina un proceso. Para estas situaciones, contamos con dos instrucciones: `for` y `while`.
- La instrucción `for` ejecuta el código entre llaves mientras la condición es verdadera. Usa la sintaxis `for(inicialización; condición; incremento)`. El primer parámetro establece los valores iniciales del bucle, el segundo parámetro es la condición que queremos comprobar y el último parámetro es una instrucción que determina cómo van a evolucionar los valores iniciales en cada ciclo.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo29.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```

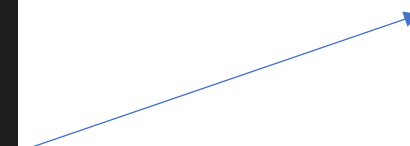


```
1  var total = 0;
2  for (var f = 0; f < 5; f++) {
3      total += 10;
4  }
5  alert("El total es: " + total); // "El total es: 50"
```

Condicionales y bucles

- Los bucles se usan frecuentemente de esta manera para hacer evolucionar el valor de una variable de acuerdo a resultados anteriores. Por ejemplo, podemos usar el bucle for para sumar todos los valores de un array.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4    <meta charset="utf-8">
5    <title>JavaScript</title>
6    <script src="micodigo30.js"></script>
7  </head>
8  <body>
9    <section>
10     <p>Hola</p>
11   </section>
12 </body>
13 </html>
```



```
1  var total = 0;
2  var lista = [23, 109, 2, 9];
3  for (var f = 0; f < 4; f++) {
4    total += lista[f];
5  }
6  alert("El total es: " + total); // "El total es: 143"
```

Condicionales y bucles

- La instrucción while solo requiere la declaración de la condición entre paréntesis y el código a ser ejecutado entre llaves. El bucle se ejecuta constantemente hasta que la condición es falsa.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo31.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```

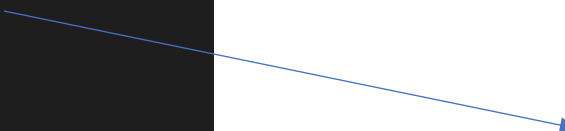


```
1  var contador = 0;
2  while(contador < 100) {
3      contador++;
4  }
5  alert("El valor es: " + contador); // "El valor es: 100"
```

Condicionales y bucles

- Si queremos que las instrucciones se ejecuten al menos una vez, sin importar cuál sea el resultado de la condición, podemos usar una implementación diferente del bucle while llamada do while. La instrucción do while ejecuta las instrucciones entre llaves y luego comprueba la condición, lo cual garantiza que las instrucciones se ejecutarán al menos una vez. La sintaxis es similar, solo tenemos que preceder las llaves con la palabra clave do y declarar la palabra clave while con la condición al final.

```
1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>JavaScript</title>
6          <script src="micodigo32.js"></script>
7      </head>
8      <body>
9          <section>
10             <p>Hola</p>
11          </section>
12      </body>
13 </html>
```

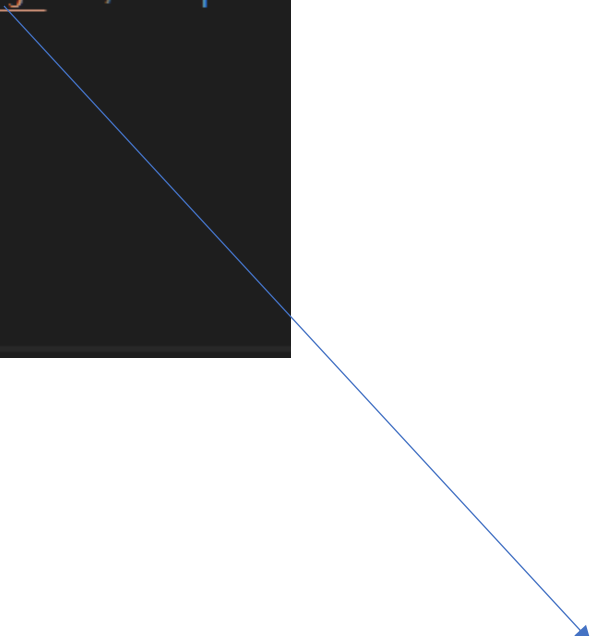


```
1  var contador = 150;
2  do {
3      contador++;
4  } while(contador < 100);
5  alert("El valor es: " + contador); // "El valor es: 151"
```

Instrucciones de transferencia de control

- Los bucles a veces se deben interrumpir. JavaScript ofrece múltiples instrucciones para detener la ejecución de bucles y condicionales. Las siguientes son las que más se usan.
 - **continue**—Esta instrucción interrumpe el ciclo actual y avanza hacia el siguiente. El sistema ignora el resto de instrucciones del bucle después de que se ejecuta esta instrucción.
 - **break**—Esta instrucción interrumpe el bucle. Todas las instrucciones restantes y los ciclos pendientes se ignoran después de que se ejecuta esta instrucción.
- La instrucción continue se aplica cuando no queremos ejecutar el resto de las instrucciones entre llaves, pero queremos seguir ejecutando el bucle.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="micodigo33.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```

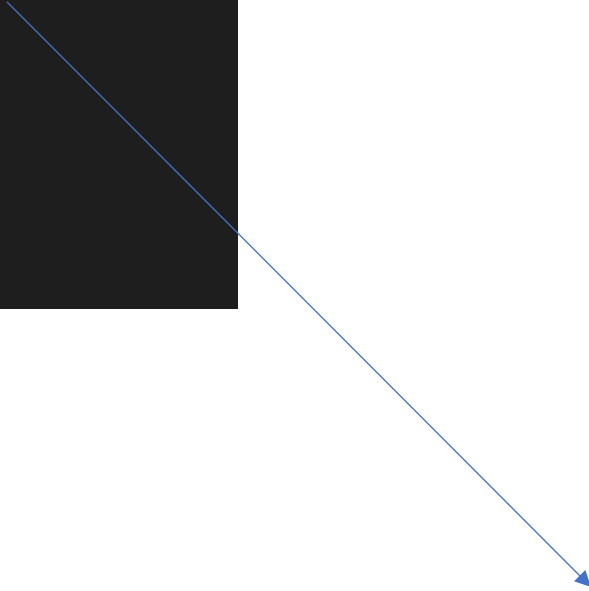


```
1  var lista = [2, 4, 6, 8];
2  var total = 0;
3  for (var f = 0; f < 4; f++) {
4      var numero = lista[f];
5      if (numero == 6) {
6          continue;
7      }
8      total += numero;
9  }
10 alert("El total es: " + total); // "El total es: 14"
```

Instrucciones de transferencia de control

- La instrucción `if` dentro del bucle `for` del ejemplo anterior compara el valor de `numero` con el valor 6. Si el valor del array que devuelve la primera instrucción del bucle es 6, se ejecuta la instrucción `continue`, la última instrucción del bucle se ignora, y el bucle avanza hacia el siguiente valor en el array `lista`. En consecuencia, todos los valores del array se suman a la variable `total` excepto el número 6.
- A diferencia de `continue`, la instrucción `break` interrumpe el bucle completamente, delegando el control a la instrucción declarada después de bucle.

```
1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>JavaScript</title>
6          <script src="micodigo34.js"></script>
7      </head>
8      <body>
9          <section>
10             <p>Hola</p>
11          </section>
12      </body>
13 </html>
```



```
1  var lista = [2, 4, 6, 8];
2  var total = 0;
3  for (var f = 0; f < 4; f++) {
4      var numero = lista[f];
5      if (numero == 6) {
6          break;
7      }
8      total += numero;
9  }
10 alert("El total es: " + total); // "El total es: 6"
```

Funciones

- Las funciones son bloques de código identificados con un nombre. La diferencia entre las funciones y los bloques de código usados en los bucles y los condicionales estudiados anteriormente es que no hay que satisfacer ninguna condición; ***las instrucciones dentro de una función se ejecutan cada vez que se llama a la función.*** Las funciones se llaman (ejecutadas) escribiendo el nombre seguido de paréntesis. Esta llamada se puede realizar desde cualquier parte del código y cada vez que sea necesario, lo cual rompe completamente el procesamiento secuencial del programa. Una vez que una función es llamada, la ejecución del programa continúa con las instrucciones dentro de la función (sin importar dónde se localiza en el código) y solo devuelve a la sección del código que ha llamado la función cuando la ejecución de la misma ha finalizado.

Declarando funciones

- Las funciones se declaran usando la palabra clave function, el nombre seguido de paréntesis, y el código entre llaves. Para llamar a la función (ejecutarla), tenemos que declarar su nombre con un par de paréntesis al final, como mostramos a continuación.

```
function mostrarMensaje() {  
    alert("Soy una función");  
}  
mostrarMensaje();
```

Declarando funciones

- ***Las funciones se deben primero declarar y luego ejecutar.*** El código siguiente declara una función llamada `mostrarMensaje()` y luego la llama una vez. Al igual que con las variables, el intérprete de JavaScript lee la función, almacena su contenido en memoria, y asigna una referencia al nombre de la función. Cuando llamamos a la función por su nombre, el intérprete comprueba la referencia y lee la función en memoria. Esto nos permite llamar a la función todas las veces que sea necesario, como los hacemos en el siguiente ejemplo.

```
1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>JavaScript</title>
6          <script src="funciones1.js"></script>
7      </head>
8      <body>
9          <section>
10             <p>Hola</p>
11          </section>
12      </body>
13 </html>
```



```
1  function mostrarMensaje() {
2      alert("Soy una función");
3  }
4  mostrarMensaje();
```

Ejemplo de funciones

- En el ejemplo siguiente se combina diferentes instrucciones ya estudiadas. Primero declara una variable y le asigna el valor 5. Luego, se declara una función llamada `calcularValores()` (pero no se ejecuta). A continuación, una instrucción `for` se usa para crear un bucle que será ejecutado mientras el valor de la variable `f` sea menor que 10. La instrucción dentro del bucle llama a la función `calcularValores()`, por lo que la función se ejecuta en cada ciclo. Cada vez que la función se ejecuta, el valor actual de `total` se multiplica por 2, duplicando su valor en cada ocasión.

```
1 <!DOCTYPE html>
2 <html lang="es">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript</title>
6     <script src="funciones2.js"></script>
7   </head>
8   <body>
9     <section>
10      <p>Hola</p>
11    </section>
12  </body>
13 </html>
```



```
1 var total = 5;
2 function calcularValores(){
3   total = total * 2;
4 }
5 for(var f = 0; f < 10; f++){
6   calcularValores();
7 }
8 alert("El total es: " + total); // "El total es: 5120"
```

Ámbito

- En JavaScript, las instrucciones que se encuentran fuera de una función se considera que están en el ámbito global. Este es el espacio en el que escribimos las instrucciones hasta que se define una función u otra clase de estructura de datos. Las variables definidas en el ámbito global tienen un alcance global y, por lo tanto, se pueden usar desde cualquier parte del código, pero las declaradas dentro de las funciones tienen un alcance local, lo que significa que solo se pueden usar dentro de la función en la que se han declarado. Esta es otra ventaja de las funciones; son lugares especiales en el código donde podemos almacenar información a la que no se podrá acceder desde otras partes del código. Esta segregación nos ayuda a evitar generar duplicados que pueden conducir a errores, como sobrescribir el valor de una variable cuando el valor anterior aún era requerido por la aplicación.
- El siguiente ejemplo ilustra cómo se definen los diferentes ámbitos y qué debemos esperar cuando accedemos desde un ámbito a variables que se han definido en un ámbito diferente.

```
1 <!DOCTYPE html>
2 <html lang="es">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript</title>
6     <script src="funciones3.js"></script>
7   </head>
8   <body>
9     <section>
10      <p>Hola</p>
11    </section>
12  </body>
13 </html>
```

```
1 var variableGlobal = 5;
2
3 function mifuncion(){
4   var variableLocal = "El valor es ";
5   alert(variableLocal + variableGlobal); // "El valor es 5"
6 }
7
8 mifuncion();
9 alert(variableLocal);
```

El código anterior declara una función llamada `mifuncion()` y dos variables, una en el ámbito global llamada `variableGlobal` y otra dentro de la función llamada `variableLocal`. Cuando se ejecuta la función, el código concatena las variables `variableLocal` y `variableGlobal`, y muestra la cadena de caracteres obtenida en la pantalla. Debido a que `variableGlobal` es una variable global, es accesible dentro de la función y, por lo tanto, su valor se agrega a la cadena de caracteres, pero cuando intentamos mostrar el valor de `variableLocal` fuera de la función, nos devuelve un error (la ventana emergente no se muestra). Esto se debe a que `variableLocal` se ha definido dentro de la función y, por lo tanto, no es accesible desde el ámbito global.

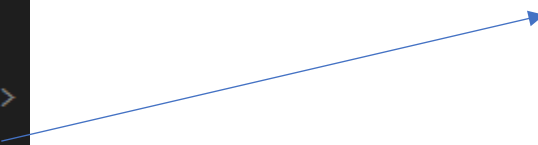
Nota

- **Lo básico:** los navegadores informan de los errores producidos por el código JavaScript en una consola oculta. Si necesita ver los errores porque genera su código, tiene que abrir esta consola desde las opciones del menú del navegador (Herramientas, en Google Chrome). Al final de este capítulo estudiaremos más estas consolas y cómo controlar errores.

Ámbito

- Debido a que las variables declaradas en diferentes ámbitos se consideran diferentes variables, dos variables con el mismo nombre, una en el ámbito global y otra en el ámbito local (dentro de una función), se considerarán dos variables distintas (se les asigna un espacio de memoria diferente).

```
1  <!DOCTYPE html>
2  <html lang="es">
3    <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="funciones4.js"></script>
7    </head>
8    <body>
9      <section>
10       <p>Hola</p>
11     </section>
12   </body>
13 </html>
```

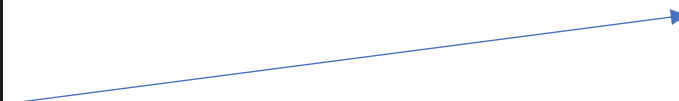


```
1  var mivariable = 5;
2  function mifuncion(){
3    var mivariable = "Esta es una variable local";
4    alert(mivariable);
5  }
6  mifuncion();
7  alert(mivariable);
```

Funciones con parámetros

- Para poder recibir un valor, la función debe incluir un nombre entre los paréntesis con el que representar el valor. Estos nombres se denominan *parámetros*. Cuando la función se ejecuta, estos parámetros se convierten en variables que podemos leer desde dentro de la función y así acceder a los valores recibidos.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="funciones5.js"></script>
7  </head>
8  <body>
9      <section>
10         <p>Hola</p>
11     </section>
12 </body>
13 </html>
```

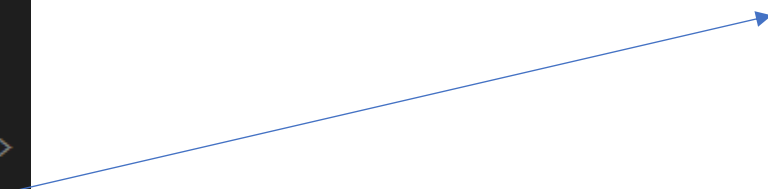


```
1  function mifuncion(valor) {
2      alert(valor);
3  }
4  mifuncion(5);
```

Funciones con parámetros

- La ventaja de usar funciones es que podemos ejecutar sus instrucciones una y otra vez, y como podemos enviar diferentes valores en cada llamada, el resultado obtenido en cada una de ellas será diferente. El siguiente ejemplo llama a la función `mifuncion()` dos veces, pero en cada oportunidad envía un valor diferente para ser procesado.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4    <meta charset="utf-8">
5    <title>JavaScript</title>
6    <script src="funciones6.js"></script>
7  </head>
8  <body>
9    <section>
10     <p>Hola</p>
11   </section>
12 </body>
13 </html>
```

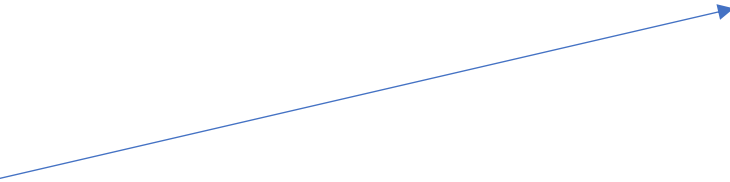


```
1  function mifuncion(valor) {
2    |    alert(valor);
3  }
4  mifuncion(5);
5  mifuncion(25);
```

Funciones con parámetros

- En este y los ejemplos anteriores, enviamos números enteros a la función, pero también podemos enviar el valor actual de una variable.

```
1 <!DOCTYPE html>
2 <html lang="es">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript</title>
6     <script src="funciones7.js"></script>
7   </head>
8   <body>
9     <section>
10      <p>Hola</p>
11    </section>
12  </body>
13 </html>
```

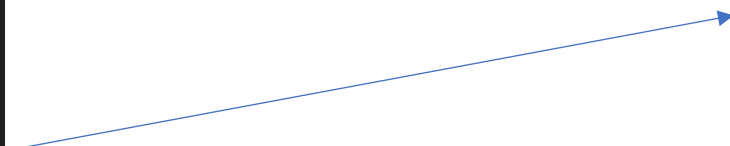


```
1 var contador = 100;
2 function mifuncion(valor) {
3   alert(valor);
4 }
5 mifuncion(contador);
```

Funciones con parámetros

- Las funciones también pueden recibir múltiples valores. Todo lo que tenemos que hacer es declarar los valores y parámetros separados por comas.

```
1 <!DOCTYPE html>
2 <html lang="es">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript</title>
6     <script src="funciones8.js"></script>
7   </head>
8   <body>
9     <section>
10      <p>Hola</p>
11    </section>
12  </body>
13 </html>
```



```
1  var contador = 100;
2  var items = 5;
3
4  function mifuncion(valor1, valor2) {
5    |    var total = valor1 + valor2;
6    alert(total); // 105
7  }
8
9  mifuncion(contador, items);
```

Funciones con retorno (return)

- El código siguiente define la misma función mifuncion() usada anteriormente, pero esta vez el valor producido por la función no se muestra en la pantalla, sino que se devuelve con la instrucción ***return***. De regreso al ámbito global, el valor devuelto por la función se asigna a la variable resultado y el contenido de esta variable se muestra en pantalla.
- ***La instrucción return finaliza la ejecución de la función.*** Cualquier instrucción declarada después de que se devuelve un valor no se ejecutará. Por esta razón, la instrucción return normalmente se declara al final de la función, pero esto no es obligatorio. Podemos devolver un valor desde cualquier parte del código si tenemos condiciones que satisfacer. Por ejemplo, la siguiente función devuelve el resultado de la suma de dos valores si el total es mayor que 100, o devuelve el valor 0 en caso contrario.

```
1 <!DOCTYPE html>
2 <html lang="es">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript</title>
6     <script src="funciones9.js"></script>
7   </head>
8   <body>
9     <section>
10      <p>Hola</p>
11    </section>
12  </body>
13 </html>
```

```
1 var contador = 100;
2 var items = 5;
3 function mifuncion(valor1, valor2) {
4   var total = valor1 + valor2;
5   return total;
6 }
7 var resultado = mifuncion(contador, items);
8 alert(resultado);
```

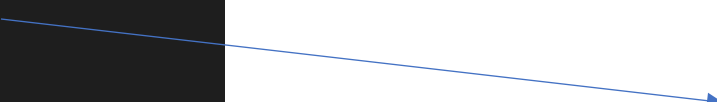
```
1 <!DOCTYPE html>
2 <html lang="es">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript</title>
6     <script src="funciones10.js"></script>
7   </head>
8   <body>
9     <section>
10      <p>Hola</p>
11    </section>
12  </body>
13 </html>
```

```
1 var contador = 100;
2 var items = 5;
3 function mifuncion(valor1, valor2) {
4   var total = valor1 + valor2;
5   if (total > 100) {
6     return total;
7   } else {
8     return 0;
9   }
10 }
11 var resultado = mifuncion(contador, items);
12 alert(resultado);
```

Funciones anónimas

- Otra manera de declarar una función es usando funciones anónimas. ***Las funciones anónimas son funciones sin un nombre o identificador. Debido a esto, se pueden pasar a otras funciones o asignar a variables. Cuando una función anónima se asigna a una variable, el nombre de la variable es el que usamos para llamar a la función, tal como hacemos en el siguiente ejemplo.***

```
1  <!DOCTYPE html>
2  <html lang="es">
3    <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script src="funciones11.js"></script>
7    </head>
8    <body>
9      <section>
10       <p>Hola</p>
11     </section>
12   </body>
13 </html>
```

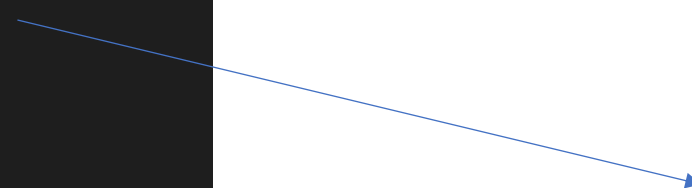


```
1  var mifuncion = function(valor) {
2    valor = valor * 2;
3    return valor;
4  };
5  var total = 2;
6  for (var f = 0; f < 10; f++) {
7    total = mifuncion(total);
8  }
9  alert("El total es " + total); // "El total es 2048"
```

Funciones anónimas

- ***Las funciones anónimas se pueden ejecutar al instante agregando paréntesis al final de su declaración.*** Esto es útil cuando queremos asignar el resultado de una operación compleja a una variable. La función procesa la operación y devuelve el resultado. En este caso, no es la función lo que se asigna a la variable, sino el valor que devuelve la misma.

```
1 <!DOCTYPE html>
2 <html lang="es">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript</title>
6     <script src="funciones12.js"></script>
7   </head>
8   <body>
9     <section>
10      <p>Hola</p>
11    </section>
12  </body>
13 </html>
```



```
1 var mivalor = function(valor) {
2   valor = valor * 2;
3   return valor;
4 }(35);
5 alert("El valor es " + mivalor); // "El valor es 70"
```

Funciones estándar

Además de las funciones que podemos crear nosotros mismos, también tenemos acceso a funciones predefinidas por JavaScript. Estas funciones realizan procesos que simplifican tareas complejas. Las siguientes son las que más se usan.

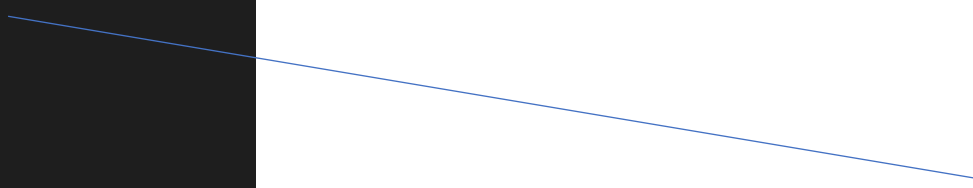
- **isNaN(valor)**—Esta función devuelve true (verdadero) si el valor entre paréntesis no es un número.
- **parseInt(valor)**—Esta función convierte una cadena de caracteres con un número en un número entero que podemos procesar en operaciones aritméticas.
- **parseFloat(valor)**—Esta función convierte una cadena de caracteres con un número en un número decimal que podemos procesar en operaciones aritméticas.
- **encodeURIComponent(valor)**—Esta función codifica una cadena de caracteres. Se utiliza para codificar los caracteres de un texto que puede crear problemas cuando se inserta en una URL.
- **decodeURIComponent(valor)**—Esta función decodifica una cadena de caracteres.

Las funciones estándar son funciones globales que podemos llamar desde cualquier parte del código; solo tenemos que llamarlas como lo hacemos con cualquier otra función con los valores que queremos procesar entre paréntesis.

isNaN()

- La función isNaN() devuelve el valor false no solo cuando la variable contiene un número, sino además cuando contiene una cadena de caracteres con un número. Esto significa que no podemos usar el valor en una operación aritmética porque podría ser una cadena de caracteres y el proceso no produciría el resultado esperado. Para asegurarnos de que el valor se puede incluir en una operación, tenemos que convertirlo en un valor numérico. Para este propósito, JavaScript ofrece dos funciones: parseInt() para números enteros y parseFloat() para números decimales.

```
1 <!DOCTYPE html>
2 <html lang="es">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript</title>
6     <script src="funciones13.js"></script>
7   </head>
8   <body>
9     <section>
10      <p>Hola</p>
11    </section>
12  </body>
13 </html>
```



```
1 var mivalor = "Hola";
2 if (isNaN(mivalor)) {
3   alert(mivalor + " no es un número");
4 } else {
5   alert(mivalor + " es un número");
6 }
```

```

1 <!DOCTYPE html>
2 <html lang="es">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript</title>
6     <script src="funciones14.js"></script>
7   </head>
8   <body>
9     <section>
10      <p>Hola</p>
11    </section>
12  </body>
13 </html>

```

```

1 var mivalor = "32";
2 if (isNaN(mivalor)) {
3   alert(mivalor + " no es un número");
4 } else {
5   var numero = parseInt(mivalor);
6   numero = numero * 10;
7   alert("El número es: " + numero); // "El número es 320"
8 }

```

```

1 <!DOCTYPE html>
2 <html lang="es">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript</title>
6     <script src="funciones15.js"></script>
7   </head>
8   <body>
9     <section>
10      <p>Hola</p>
11    </section>
12  </body>
13 </html>

```

Otra función estándar útil es `encodeURIComponent()`. Con esta función podemos preparar una cadena de caracteres para ser incluida en una URL. El problema con las URL es que le otorgan un significado especial a algunos caracteres, como `?` o `&`. Debido a que los usuarios no conocen estas restricciones, tenemos que codificar las cadenas de caracteres antes de incluirlas en una URL cada vez que las introduce el usuario o provienen de una fuente que no es fiable.

```

1 var nombre = "Juan Perez";
2 var codificado = encodeURIComponent(nombre);
3 var miURL = "http://www.ejemplo.com/contacto.html?nombre=" + codificado;
4 alert(miURL);

```